

Sistemas Distribuídos – Engenharia de Telecomunicações

Tipos e arquiteturas

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](https://creativecommons.org/licenses/by/4.0/)

Tipos de Sistemas Distribuídos

Sistemas de computação distribuídos

Ambientes para processamento paralelo

- **SMP (*Symmetric Multiprocessing*)**

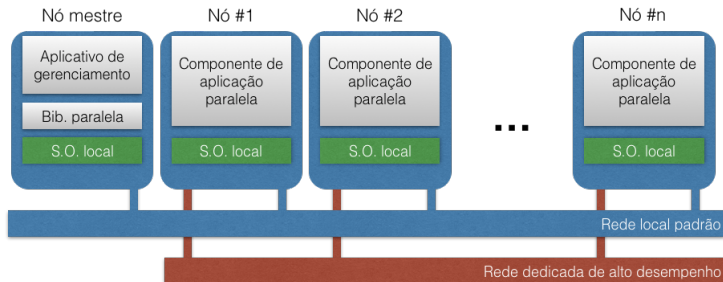
- Processadores compartilham memória e recursos
- Comunicação via memória compartilhada
- Sistema operacional único

- **Cluster ou MPP (*Massively Parallel Processing*)**

- Processadores com recursos dedicados
- Comunicação via rede de alta velocidade
- Sistema operacional próprio por nó
- Ambiente *shared-nothing*
- Clusters podem ser fisicamente dispersos, enquanto MPP normalmente está em um único gabinete

Cluster: agrupamento de máquinas

- Aglomerado de computadores semelhantes
 - Mesmo sistema operacional e conectados através de uma rede local
 - O administrador é dono de todo do cluster
- Usado para processamento paralelo – no primário distribui tarefas para os demais nós



Computação de Alto Desempenho

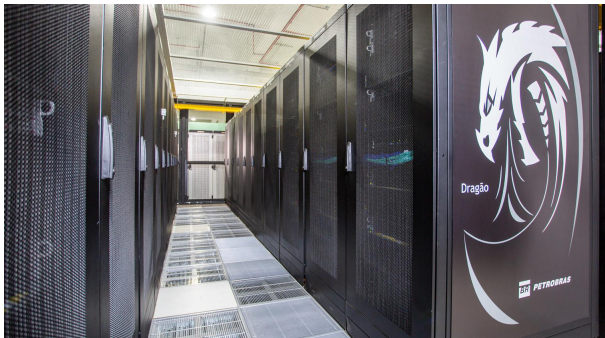
(High-performance computing - HPC)



LineShine - China - O mais rápido jun/2026 – 2.198 Exaflop/s
13.789.440 núcleos. Fonte: <https://top500.org>

Computação de Alto Desempenho

(High-performance computing - HPC)



Crédito: Rafael Wallace

Dragão (Petrobras) - Brasil

Em 06/2022 estava na 60ª posição com 8,98 PFlop/s

188.224 núcleos

Computação de Alto Desempenho

(High-performance computing - HPC)



Santos Dumont - Brasil

Em junho/2022 estava na 424ª com 5,1 PFlop/s

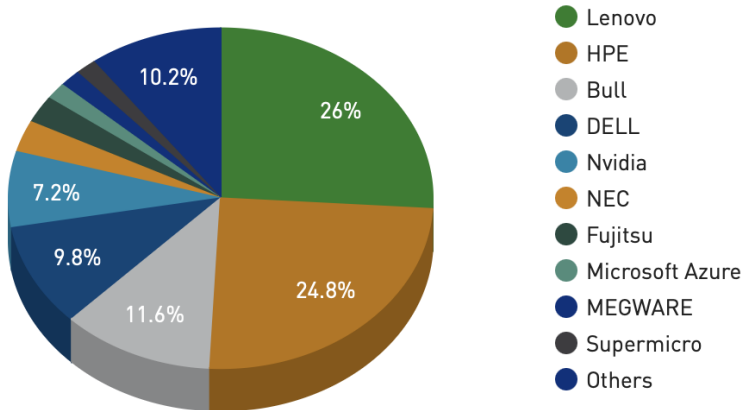
36.432 núcleos, 1.134 nós computacionais

<http://sdumont.lncc.br>

Computação de Alto Desempenho

(High-performance computing - HPC)

Vendors System Share

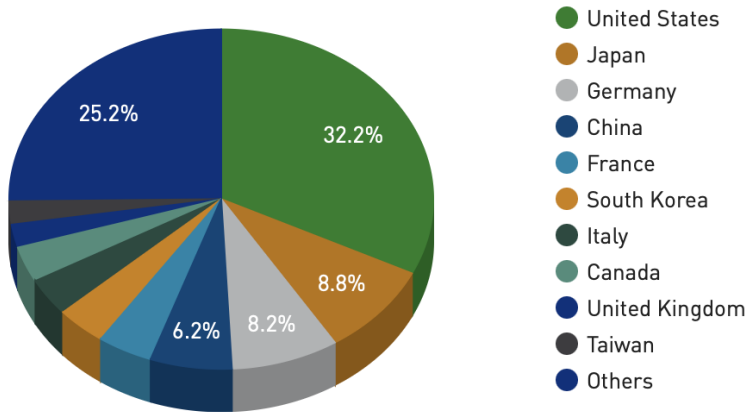


Fonte: <https://top500.org>. Junho de 2026

Computação de Alto Desempenho

(High-performance computing - HPC)

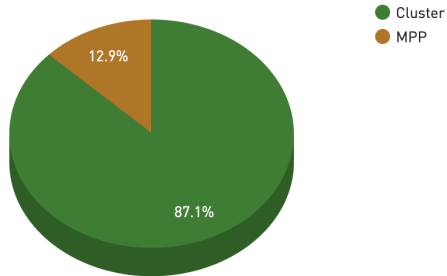
Countries System Share



Computação de Alto Desempenho

(High-performance computing - HPC)

Architecture System Share



- **Processamento Massivamente Paralelo (MPP)**
 - Um único sistema com muitos processadores
- **Cluster**
 - Um conjunto de computadores interconectados que trabalham juntos como um único sistema

Computação de Alto Desempenho

(High-performance computing - HPC)



Cluster Beowulf - simples, porém acessível

Grades computacionais ou Grid

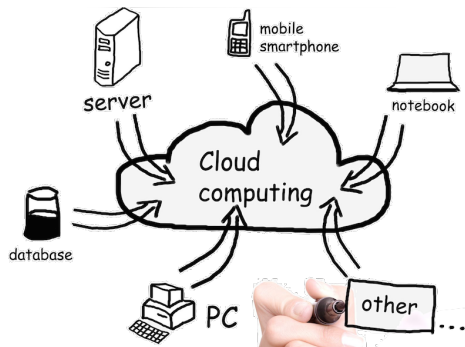
Grid

Ambientes heterogêneos: cada nó pode ter um hardware, S.O., rede, domínio administrativo e políticas de segurança diferentes

- *Middleware* é responsável por prover abstração para esta heterogeneidade
- Grids geralmente são usadas para constituir **Organizações Virtuais**
 - Diversas instituições colaboram com seus recursos (*cluster*, armazenamento, etc.)
 - Políticas de uso determinam quanto tempo cada organização poderá usar a grid

Cloud Computing ou Computação em Nuvem

- Permite que **recursos computacionais** possam ser **providos de maneira rápida** e com **mínimo esforço de gerenciamento** por parte do provedor de serviço
- Permite adicionar ou remover recursos de acordo com a demanda
 - Similar as redes de energia e de telefonia



Cloud Computing ou Computação em Nuvem

- **Ofere como serviço** aplicações, bem como softwares e hardwares que hospedam estas aplicações
 - A “nuvem” é conjunto de hardware e software no *datacenter*
- **Modelo de implantação**
 - **Nuvem privada**
 - Toda a nuvem é construída dentro dos limites de uma instituição e é voltada exclusivamente para seus usuários
 - **Nuvem pública**
 - Oferece os serviços e a tarifação é feita de acordo com o uso
 - A nuvem armazena dados de diferentes clientes

Cloud Computing ou Computação em Nuvem

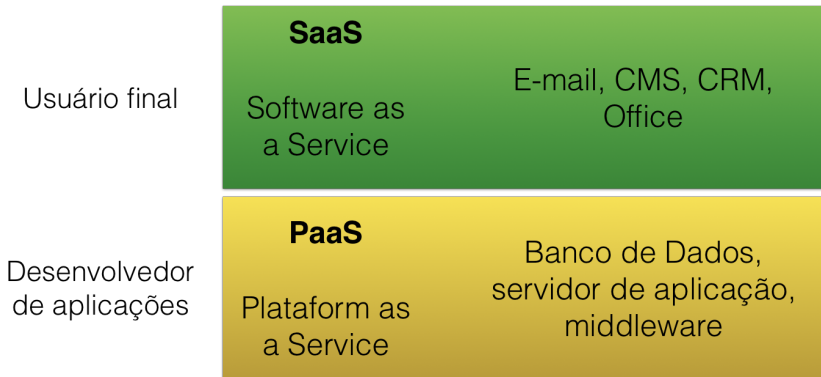
Usuário final

SaaS

Software as
a Service

E-mail, CMS, CRM,
Office

Cloud Computing ou Computação em Nuvem



Cloud Computing ou Computação em Nuvem

Usuário final	SaaS Software as a Service E-mail, CMS, CRM, Office
Desenvolvedor de aplicações	PaaS Platform as a Service Banco de Dados, servidor de aplicação, middleware
Analista de redes	IaaS Infrastructure as a Service Hardware em geral

Cloud Computing ou Computação em Nuvem

Usuário final	SaaS Software as a Service	Gmail, Office 365, Adobe Creative Suite, ShareLaTeX
Desenvolvedor de aplicações	PaaS Plataform as a Service	Google App Engine, Microsoft Azure, Heroku
Analista de redes	IaaS Infrastructure as a Service	Amazon AWS, Microsoft Azure, Google Compute Engine, Rackspace

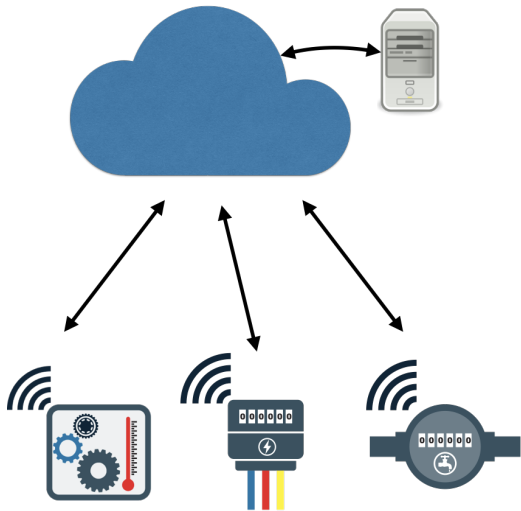
Fog computing – computação em névoa



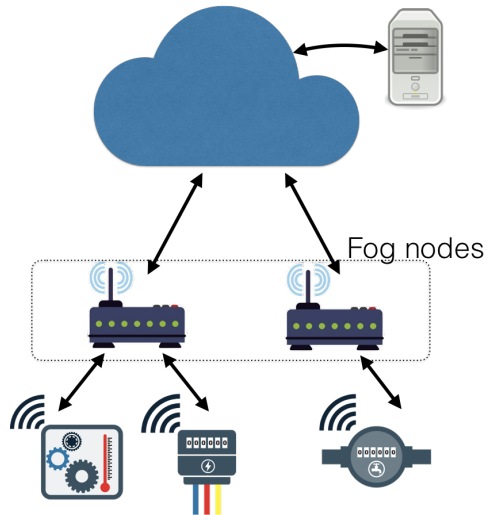
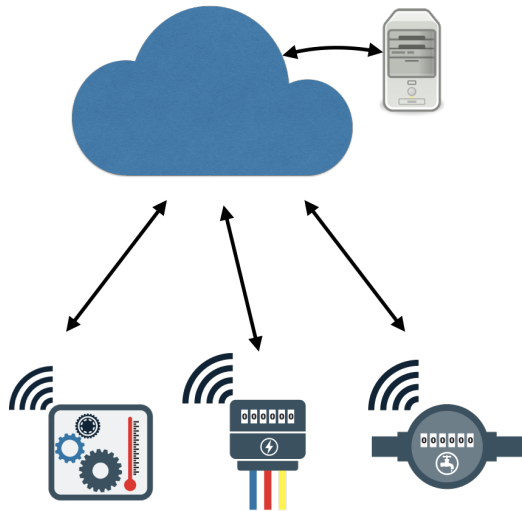
Fog computing – computação em névoa



Fog computing – computação em névoa

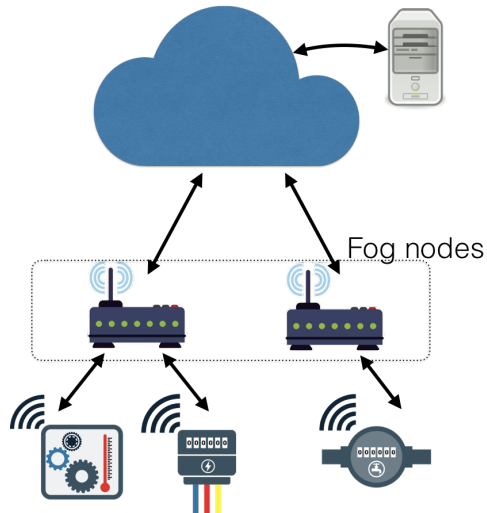


Fog computing – computação em névoa



Fog computing – computação em névoa

- Poder do processamento mais perto do limite da borda da rede
- Dados, computação, medição são distribuídos e colocados nos locais mais adequados e eficientes
- Redução da quantidade de dados transmitidos



Sistemas de informação distribuídos

Processamento de Transações



MONTE SUA VIAGEM

☒ Voo
☐ Hotel
☐ Carro
☐ Atracões

☐ Voo + Hotel
☐ Voo + Carro
☐ Voo + Hotel + Carro
☐ Hotel + Carro

Voo

☒ Ida e volta ☐ Somente ida ☐ Vários Destinos

Saindo de: Partida: Horário:

Indo para: Retorno: Horário:

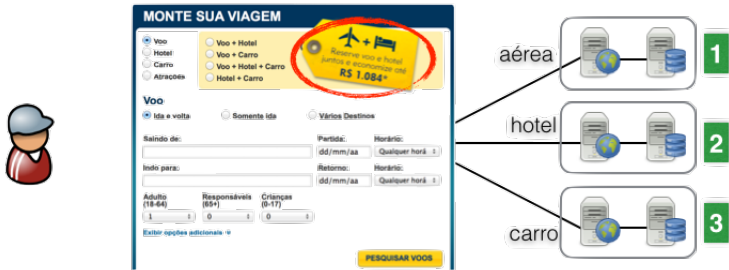
Adulto (18-64) Responsáveis (65+) Crianças (0-17)

[Exibir opções adicionais](#)

PERQUISAR VOOS

Reserve voo e hotel juntos e economize até R\$ 1.084*

Processamento de Transações



Processamento de Transações



Sistema do hotel indisponível, ou o hotel não tem mais vaga (acesso concorrente de diversos clientes)

Processamento de Transações



ou tudo, ou nada

MONTE SUA VIAGEM

☒ Voo ☐ Hotel ☐ Carro ☐ Atracões

☐ Voo + Hotel ☐ Voo + Carro ☐ Voo + Hotel + Carro ☐ Hotel + Carro

Voo

☒ Ida e volta ☐ Somente ida ☐ Vários Destinos

Saindo de: Partida: dd/mm/aa Horário: Qualquer hora

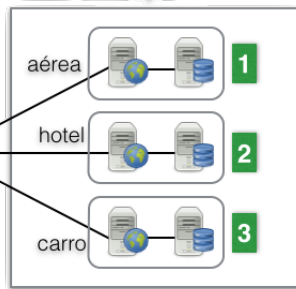
Indo para: Retorno: dd/mm/aa Horário: Qualquer hora

Adulto (18-64) Responsáveis (65+) Crianças (0-17)

[Exibir opções adicionais](#)

PESQUISAR VOOS

inicia transação



finaliza transação

Processamento de Transações

- **Transações** tem por objetivo **garantir a integridade de recursos**
 - Recurso acessado por diversos processos de forma concorrente
 - Processo realizando operações sobre um ou mais recursos
- **Modo de operação** das transações
 - 1 Indicar início da transação
 - 2 Realizar operações sobre os recursos
 - 3 Indicar o término da transação
 - *commit* – tenta salvar as alterações
 - *abort* – ignora as alterações e volta para os valores originais

Ou todas as operações na transação são executadas ou nenhuma delas será executada

Características das Transações

- **Atômica**
 - Para um observador externo, a transação é indivisível
 - É executada em sua totalidade ou não executa
- **Consistente**
 - Toda transição leva o sistema de um estado válido para um outro estado válido
- **Isolada**
 - Transações concorrentes não geram interferência entre si
- **Duráveis**
 - Uma vez que a transação foi concluída com sucesso (commit), então as modificações feitas por ela são permanentes

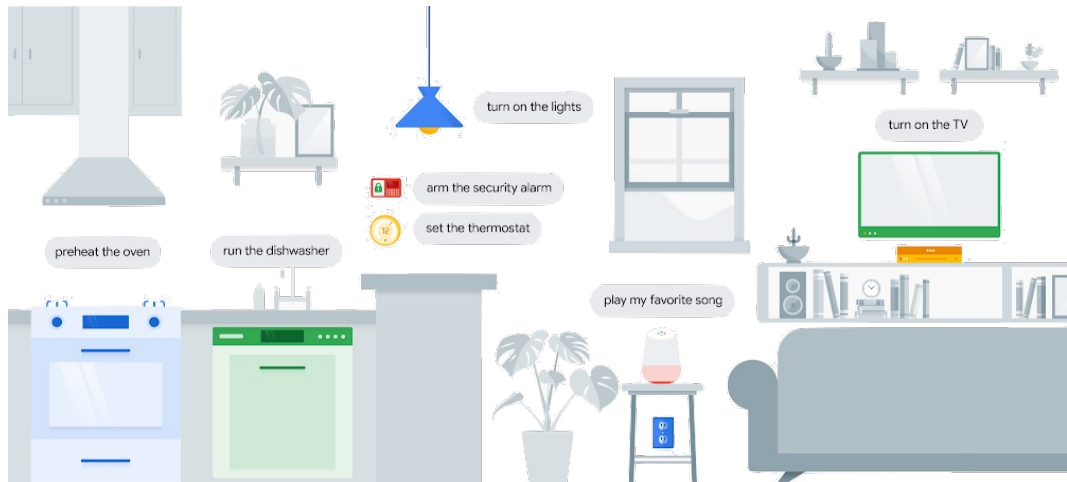
Sistemas de informação distribuídos

Sistemas distribuídos ubíquos

Sistemas distribuidos ubíquos – IoT



Sistemas distribuídos ubíquos – IoT



Sistemas distribuídos ubíquos

- Dispositivos móveis pequenos, alimentados por bateria e com conexão sem fio
 - Ex: Laptop, tablet, celular, dispositivos de automação residencial ou para saúde, rede de sensores sem fio
- Requisitos para sistemas ubíquos
 - **Reagir a mudanças do contexto**
 - O dispositivo deve estar continuamente ciente das mudanças no ambiente
 - **Encorajar composições *ad hoc***
 - Deve ser fácil cooperar com outros dispositivos
 - **Compartilhamento de recursos é algo implícito**
 - Prover facilidades para compartilhar informações

Arquiteturas de Sistemas Distribuídos

Arquiteturas de Sistemas Distribuídos

Organização dos Sistemas Distribuídos

- **Estilos de arquitetura** – formas de projetar o software do SD
 - Camadas
 - Objetos
 - Eventos
 - Espaço de dados compartilhados
- **Arquitetura de sistema** – organização do SD em máquinas reais
 - Centralizados
 - Descentralizados
 - Híbridos

Arquitetura de software

Estilos de arquitetura

Estilos são definidos por **componentes** de software (processos) e como estes se **conectam** uns aos outros

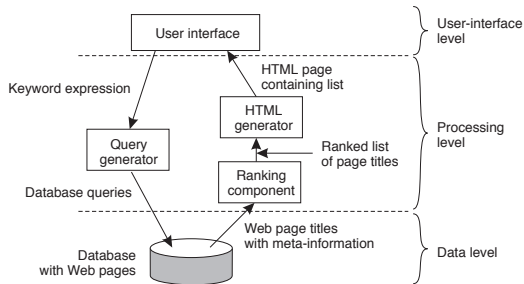
- Baseada em camadas
- Baseada em objetos
- Baseada em eventos
- Espaço de dados compartilhados

Arquitetura de software

Estilos de arquitetura

Estilos são definidos por **componentes** de software (processos) e como estes se **conectam** uns aos outros

- Baseada em camadas
- Baseada em objetos
- Baseada em eventos
- Espaço de dados compartilhados



Processos das camadas superiores invocam processos das camadas inferiores

Semelhante à pilha TCP/IP

Arquitetura de software

Estilos de arquitetura

Estilos são definidos por **componentes** de software (processos) e como estes se **conectam** uns aos outros

- Baseada em camadas
- Baseada em objetos
- Baseada em eventos
- Espaço de dados compartilhados
- Cada objeto contém um **estado** e é capaz de **realizar operações** que podem ser **invocadas por outros objetos**
- O **objeto** seria um **componente** de software
- Os **conectores** entre os objetos podem ser representadas por **chamadas de procedimento remoto** (RPC)

Arquitetura de software

Estilos de arquitetura

Estilos são definidos por **componentes** de software (processos) e como estes se **conectam** uns aos outros

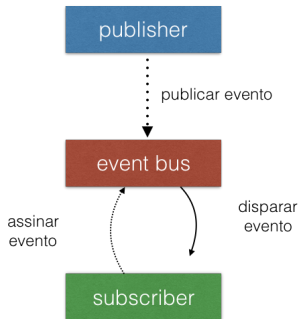
- Baseada em camadas
- Baseada em objetos
- Baseada em eventos
- Espaço de dados compartilhados
- **Comunicação** entre os processos é feita **através de eventos**, que podem ou não conter dados
- O barramento de eventos (*middleware*) é o responsável pelo transporte
- Ex: Sistema *publish/subscribe*
 - Processos registram o interesse em determinados eventos (*subscribe*)
 - Processos publicam eventos
 - *Middleware* notifica somente os processos que assinaram aquele evento

Arquitetura de software

Estilos de arquitetura

Estilos são definidos por **componentes** de software (processos) e como estes se **conectam** uns aos outros

- Baseada em camadas
- Baseada em objetos
- Baseada em eventos
- Espaço de dados compartilhados



desacoplamento espacial – um processo não precisa referenciar diretamente um outro

Arquitetura de software

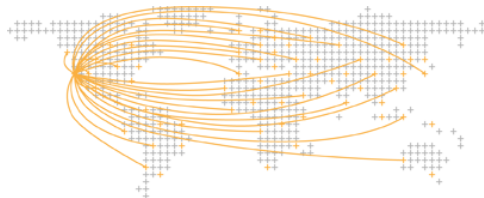
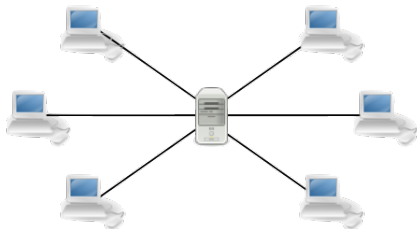
Estilos de arquitetura

Estilos são definidos por **componentes** de software (processos) e como estes se **conectam** uns aos outros

- Baseada em camadas
- Baseada em objetos
- Baseada em eventos
- Espaço de dados compartilhados
- **Semelhante** ao baseado em **eventos**, porém os **dados persistem** no *middleware*
- Acontece também o **desacoplamento temporal**
 - Dados persistem, mesmo se os processos não estiverem mais ativos

Arquitetura Centralizada – modelo cliente-servidor

- Aplicação é modelada como um conjunto de serviço que são providos pelos **servidores** e um conjunto de **clientes** que acessam esses serviços
 - Cliente envia um pedido ao servidor e aguarda por sua resposta
 - Clientes precisam conhecer os servidores, porém o inverso não é verdade
- Em um sistema distribuído, é possível que um servidor também seja cliente de um outro servidor

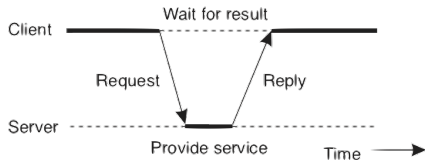
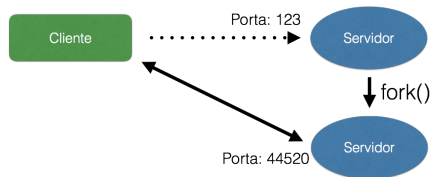


Fonte: Diaspora

Arquitetura Centralizada – modelo cliente-servidor

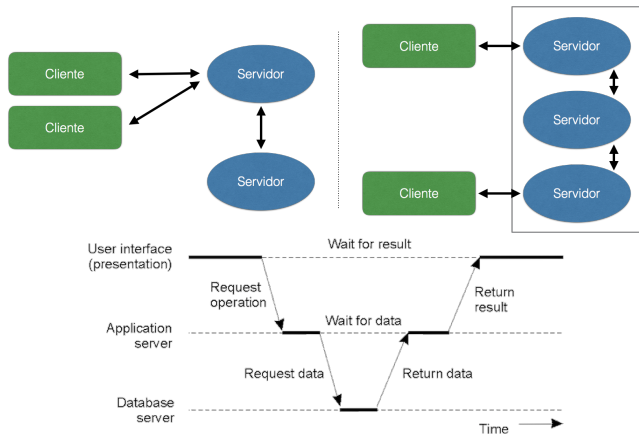
Problemas com múltiplos clientes e um único servidor

Servidor pode se tornar um gargalo e ponto único de falhas



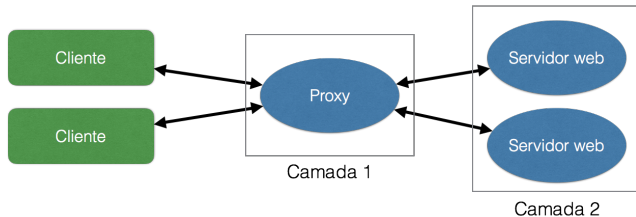
Arquitetura Centralizada – modelo cliente-servidor

Solução: Múltiplos servidores



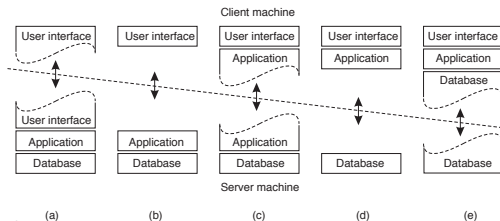
Arquitetura Centralizada – modelo cliente-servidor

Múltiplas camadas – Servidor torna-se cliente de outro servidor



Arquitetura de sistema

Centralizada – modelo cliente-servidor



- **Clientes leves – *thin client***

- Maior facilidade para manter
 - Aplicar atualização
- Pior desempenho
 - depende da latência da rede

- **Clientes pesados – *fat client***

- Dificuldade para manter
 - Diferentes versões de S.O e conflitos com outros softwares
- Melhor desempenho
 - Pouca dependência da rede

- 1 Qual destas seria mais fácil manter?
- 2 Qual destas está em evidência atualmente?

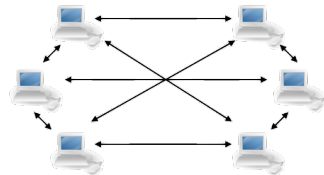
Arquitetura Descentralizada – (Peer-to-Peer – P2P)

- **Características**

- Todo nó na rede possui a mesma funcionalidade
 - Pode acessar serviços de outros nós, bem como prover
- Nós são autônomos e não confiáveis
- Nós ingressam e saem da rede a qualquer instante

- **Motivação**

- Evitar ponto único de falhas
- Escalabilidade
- Aproveitar recursos de todos os nós participantes da rede



Fonte: Diaspora

Arquitetura Descentralizada – (*Peer-to-Peer* – P2P)

Potenciais benefícios

- Redes P2P são comumente utilizadas para distribuição de conteúdo
- Escala para um grande número de nós
- Organização dinâmica diante de ingressos e egressos na rede
- Balanceamento de carga
- Processamento em paralelo
- Alta disponibilidade por meio de replicação dos recursos nos nós

Arquitetura Descentralizada – (*Peer-to-Peer* – P2P)

Topologias

- **Não estruturadas**

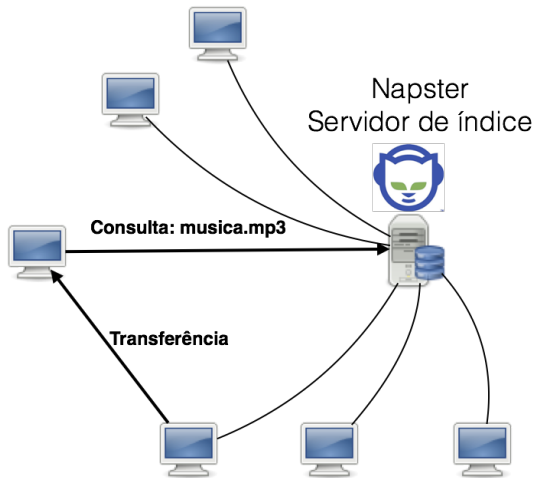
- Todos os nós na rede possuem as mesmas funcionalidades
- Exemplos: Gnutella, Napster, Kazaa (com super-nós), redes como Bitcoin¹

- **Estruturadas**

- Não existe um nó central, mas um conjunto de nós é responsável por receber consultas sobre recursos e facilitar o acesso esses
- Exemplos: Chord, [Diaspora](#) e [Mastodon](#)

¹Veja em <https://bitnodes.io> uma estimativa do número de nós que participam da rede P2P

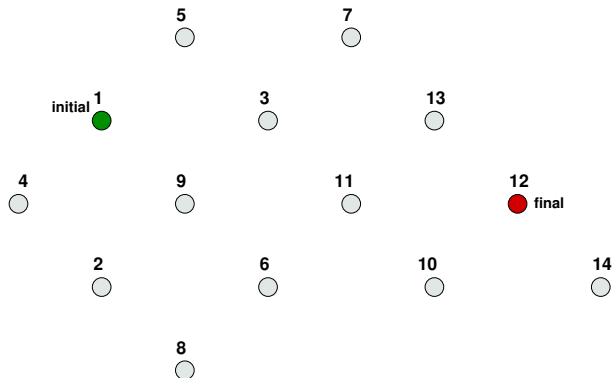
Redes P2P não estruturadas – Busca por conteúdo



- Arquivos são mantidos em cada nó da rede
- Lista de arquivos são enviados para o servidor central do Napster
- Servidor retorna a lista de IPs que possuem o arquivo
- Transferência de arquivos entre 2 clientes

Busca por conteúdo – Ex: Gnutella

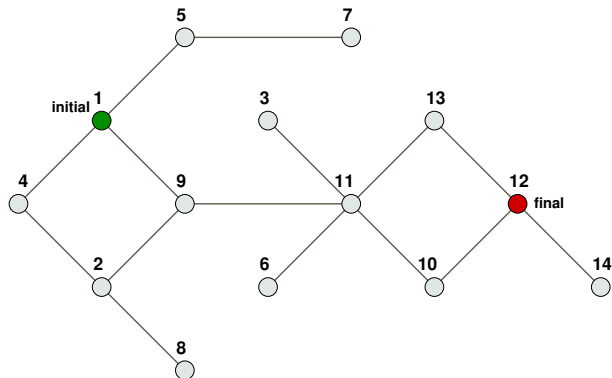
- Cada nó possui um **índice parcial** que representa um subconjunto de todos os nós participantes da rede – **não existe nó central**
 - **Lei de potência** – muitos nós possuem poucos vizinhos e poucos nós possuem muitos vizinhos (conceito de Mundo Pequeno)



- Busca por inundação (*flood*)

Busca por conteúdo – Ex: Gnutella

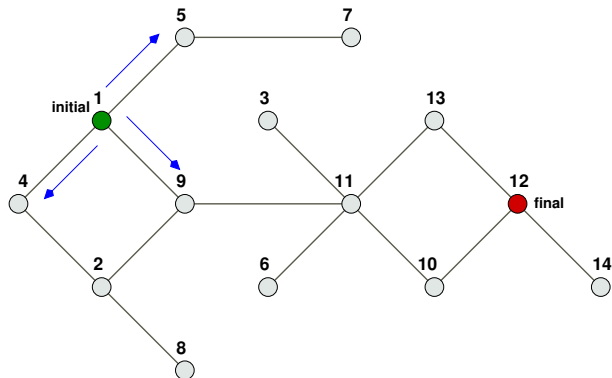
- Cada nó possui um **índice parcial** que representa um subconjunto de todos os nós participantes da rede – **não existe nó central**
 - **Lei de potência** – muitos nós possuem poucos vizinhos e poucos nós possuem muitos vizinhos (conceito de Mundo Pequeno)



- Busca por inundação (*flood*)

Busca por conteúdo – Ex: Gnutella

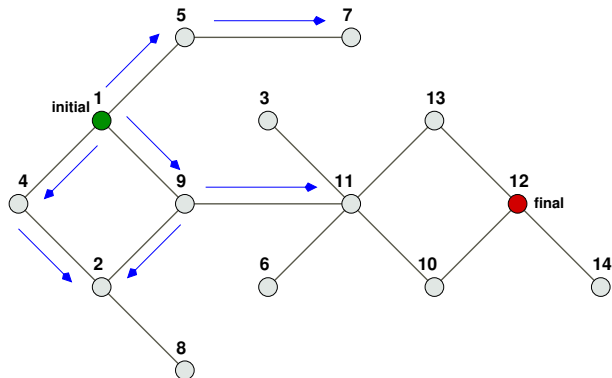
- Cada nó possui um **índice parcial** que representa um subconjunto de todos os nós participantes da rede – **não existe nó central**
 - **Lei de potência** – muitos nós possuem poucos vizinhos e poucos nós possuem muitos vizinhos (conceito de Mundo Pequeno)



- Busca por inundação (*flood*)

Busca por conteúdo – Ex: Gnutella

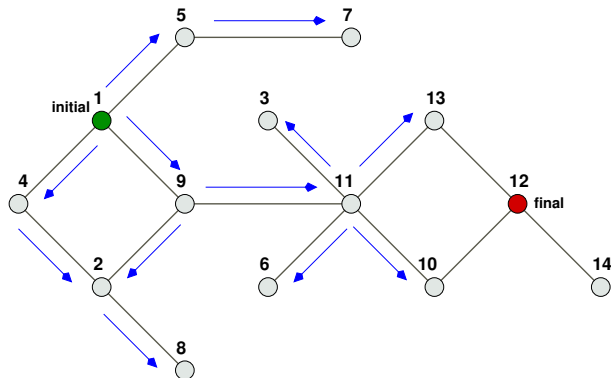
- Cada nó possui um **índice parcial** que representa um subconjunto de todos os nós participantes da rede – **não existe nó central**
 - **Lei de potência** – muitos nós possuem poucos vizinhos e poucos nós possuem muitos vizinhos (conceito de Mundo Pequeno)



- Busca por inundação (*flood*)

Busca por conteúdo – Ex: Gnutella

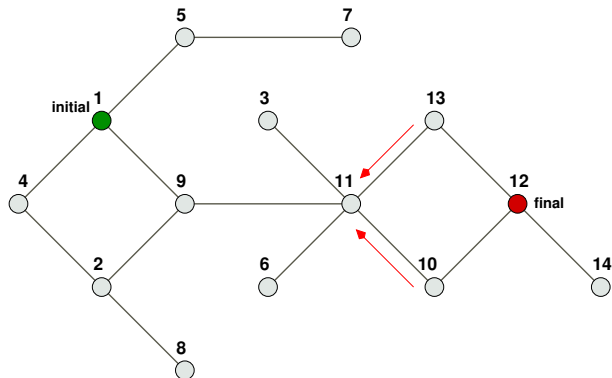
- Cada nó possui um **índice parcial** que representa um subconjunto de todos os nós participantes da rede – **não existe nó central**
 - **Lei de potência** – muitos nós possuem poucos vizinhos e poucos nós possuem muitos vizinhos (conceito de Mundo Pequeno)



- Busca por inundação (*flood*)

Busca por conteúdo – Ex: Gnutella

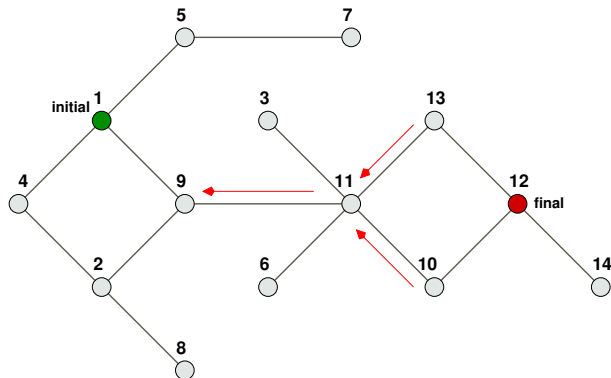
- Cada nó possui um **índice parcial** que representa um subconjunto de todos os nós participantes da rede – **não existe nó central**
 - **Lei de potência** – muitos nós possuem poucos vizinhos e poucos nós possuem muitos vizinhos (conceito de Mundo Pequeno)



- Busca por inundação (*flood*)

Busca por conteúdo – Ex: Gnutella

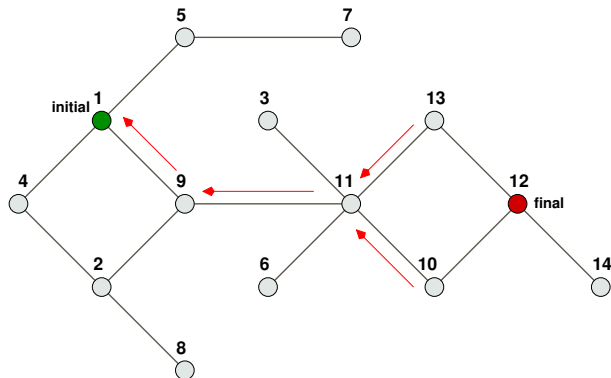
- Cada nó possui um **índice parcial** que representa um subconjunto de todos os nós participantes da rede – **não existe nó central**
 - **Lei de potência** – muitos nós possuem poucos vizinhos e poucos nós possuem muitos vizinhos (conceito de Mundo Pequeno)



- Busca por inundação (*flood*)

Busca por conteúdo – Ex: Gnutella

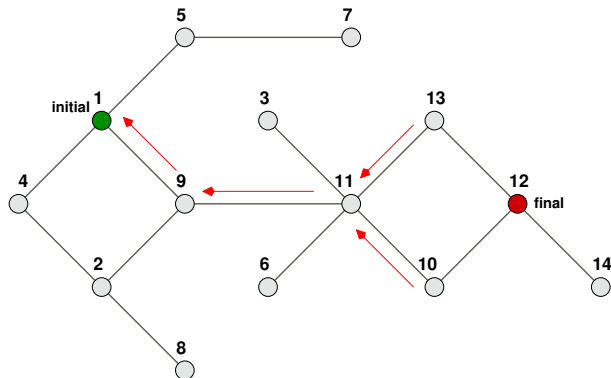
- Cada nó possui um **índice parcial** que representa um subconjunto de todos os nós participantes da rede – **não existe nó central**
 - **Lei de potência** – muitos nós possuem poucos vizinhos e poucos nós possuem muitos vizinhos (conceito de Mundo Pequeno)



- Busca por inundação (*flood*)

Busca por conteúdo – Ex: Gnutella

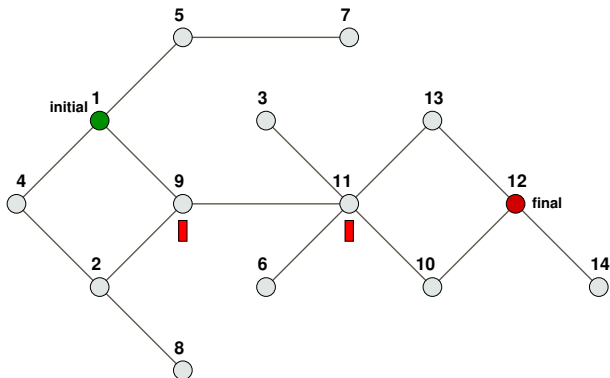
- Cada nó possui um **índice parcial** que representa um subconjunto de todos os nós participantes da rede – **não existe nó central**
 - **Lei de potência** – muitos nós possuem poucos vizinhos e poucos nós possuem muitos vizinhos (conceito de Mundo Pequeno)



- Busca por inundação (*flood* + tabela cache)

Busca por conteúdo – Ex: Gnutella

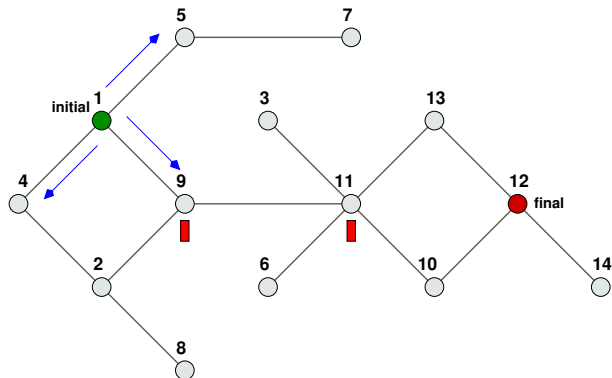
- Cada nó possui um **índice parcial** que representa um subconjunto de todos os nós participantes da rede – **não existe nó central**
 - **Lei de potência** – muitos nós possuem poucos vizinhos e poucos nós possuem muitos vizinhos (conceito de Mundo Pequeno)



- Busca por inundação (*flood* + tabela cache)

Busca por conteúdo – Ex: Gnutella

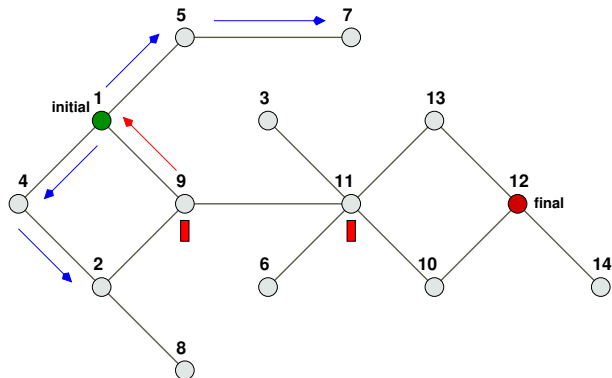
- Cada nó possui um **índice parcial** que representa um subconjunto de todos os nós participantes da rede – **não existe nó central**
 - **Lei de potência** – muitos nós possuem poucos vizinhos e poucos nós possuem muitos vizinhos (conceito de Mundo Pequeno)



- Busca por inundação (*flood* + tabela cache)

Busca por conteúdo – Ex: Gnutella

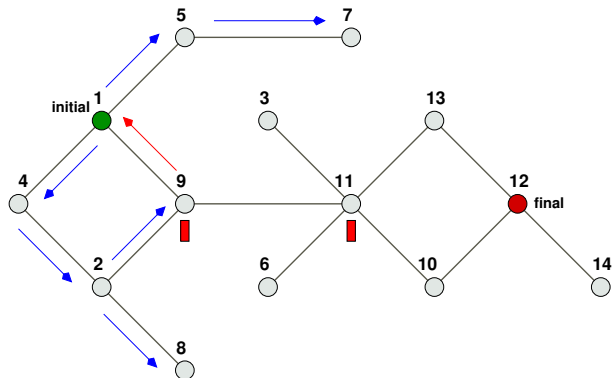
- Cada nó possui um **índice parcial** que representa um subconjunto de todos os nós participantes da rede – **não existe nó central**
 - **Lei de potência** – muitos nós possuem poucos vizinhos e poucos nós possuem muitos vizinhos (conceito de Mundo Pequeno)



- Busca por inundação (*flood* + tabela cache)

Busca por conteúdo – Ex: Gnutella

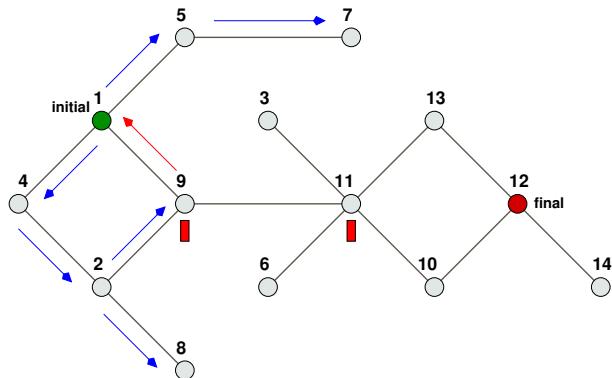
- Cada nó possui um **índice parcial** que representa um subconjunto de todos os nós participantes da rede – **não existe nó central**
 - **Lei de potência** – muitos nós possuem poucos vizinhos e poucos nós possuem muitos vizinhos (conceito de Mundo Pequeno)



- Busca por inundação (*flood* + tabela cache)

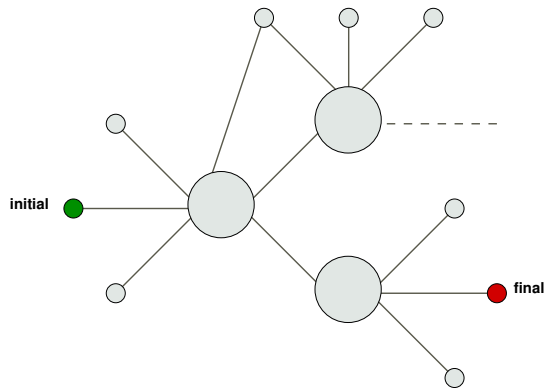
Busca por conteúdo – Ex: Gnutella

- Cada nó possui um **índice parcial** que representa um subconjunto de todos os nós participantes da rede – **não existe nó central**
 - **Lei de potência** – muitos nós possuem poucos vizinhos e poucos nós possuem muitos vizinhos (conceito de Mundo Pequeno)



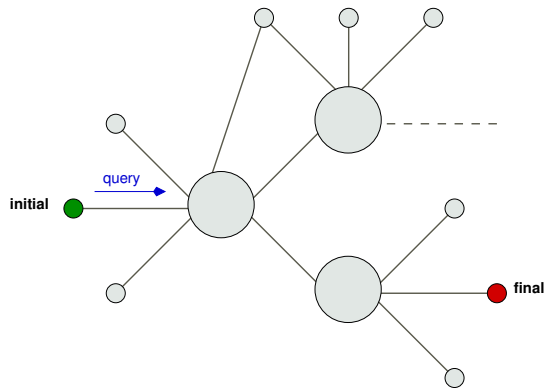
- Protocolos epidêmicos são mais eficientes. Ex: gossip

Redes P2P não estruturadas com Ultrapeers – Ex: Kazaa



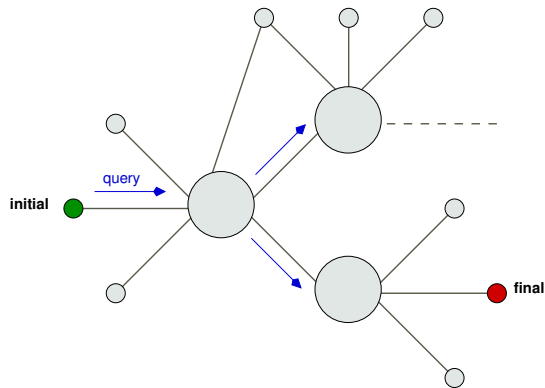
- Busca por inundação é muito custosa
- Supernós concentram índices de seus vizinhos

Redes P2P não estruturadas com Ultrapeers – Ex: Kazaa



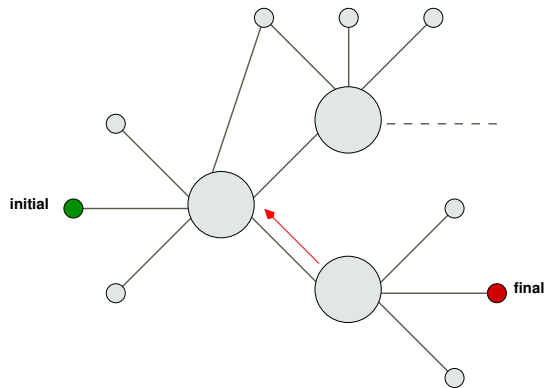
- Busca por inundação é muito custosa
- Supernós concentram índices de seus vizinhos

Redes P2P não estruturadas com Ultrapeers – Ex: Kazaa



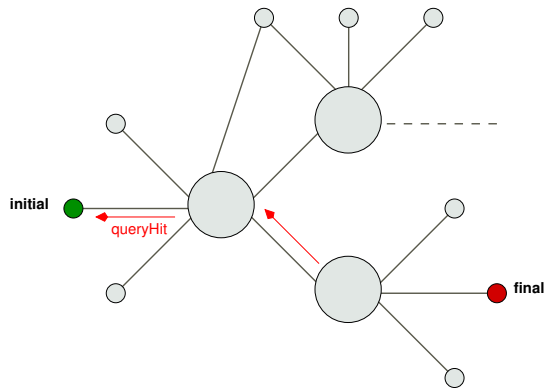
- Busca por inundação é muito custosa
- Supernós concentram índices de seus vizinhos

Redes P2P não estruturadas com Ultrapeers – Ex: Kazaa



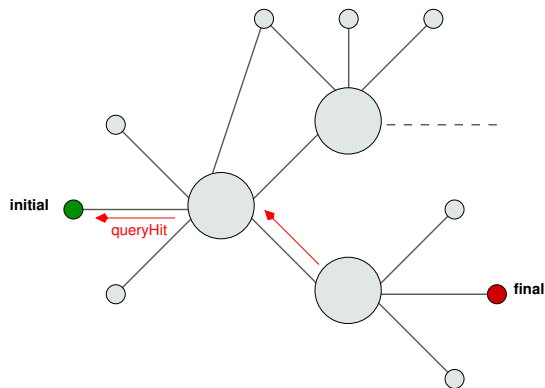
- Busca por inundação é muito custosa
- Supernós concentram índices de seus vizinhos

Redes P2P não estruturadas com Ultrapeers – Ex: Kazaa



- Busca por inundação é muito custosa
- Supernós concentram índices de seus vizinhos

Redes P2P não estruturadas com Ultrapeers – Ex: Kazaa



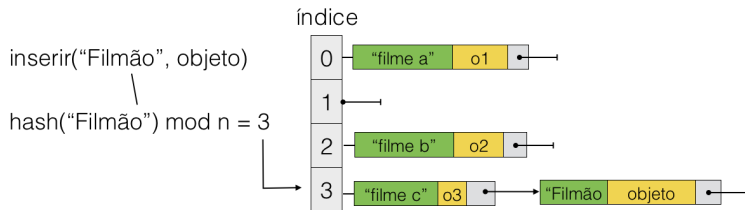
- Busca por inundação é muito custosa
- Supernós concentram índices de seus vizinhos

- Temos aqui duas redes sobrepostas: uma para os supernós e outra para os nós comuns
- Como determinar quem será supernó?

Tabelas de dispersão (*Hash table*)

- **Função hash** sobre um valor (V) para encontrar a posição (índice) onde será armazenado
 - Para buscar: Dado uma chave (K), retornará um valor (V)

```
Map<String, Filme> catalogo = new HashMap<>();  
Filme objeto = new Filme("Titulo");  
catalogo.put("filmeão", objeto);  
  
Filme res = catalogo.get("filmeão");
```

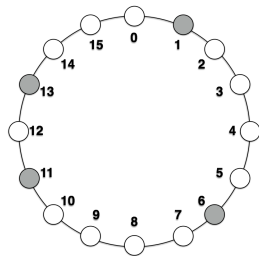


Redes P2P estruturadas – *Distributed Hash Table (DHT)*

- Algoritmo de hash serve para determinar em qual nó (índice) será armazenado um valor
 - Mapeia chaves (K) $\Rightarrow$ Nós
- Apresentam mecanismos que controlam a topologia da rede e distribuem os recursos por nós específicos
 - Adequado para ambientes de larga escala
- **Buscam** tendem a ser **mais eficientes** do que nas redes não estruturadas
- Aplicações que usam DHT
 - Tracker distribuído do BitTorrent, eDonkey, Freenet

Chord – protocolo e algoritmo para P2P DHT

- Chaves (K) dos recursos são atribuídas aos nós
- Cada nó ocupa uma posição no círculo e mantém um ponteiro para seu sucessor
 - posição= $\text{hash}(IP) \bmod n$
 - Na figura uma rede com 4 nós (1, 6, 11 e 13)

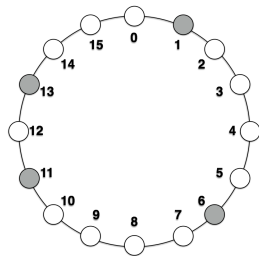


Sucessor

- O sucessor de um identificador é o primeiro nó no sentido horário cujo ID é igual ou sucede o identificador

Chord – protocolo e algoritmo para P2P DHT

- Chaves (K) dos recursos são atribuídas aos nós
- Cada nó ocupa uma posição no círculo e mantém um ponteiro para seu sucessor
 - posição= $\text{hash}(IP) \bmod n$
 - Na figura uma rede com 4 nós (1, 6, 11 e 13)



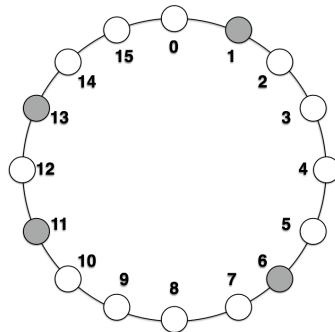
Sucessor

- O sucessor de um identificador é o primeiro nó no sentido horário cujo ID é igual ou sucede o identificador
- Quais os sucessores dos Ids: 12, 1, 4 e 8?

Chord

Cada nó mantém um ponteiro para o seu nó sucessor

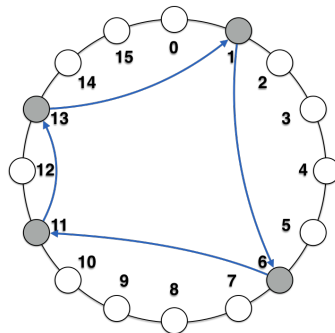
- O sucessor do nó n é $\text{sucessor}(n+1)$
 - O sucessor de 1 é:
 - O sucessor de 6 é:
 - O sucessor de 11 é:
 - O sucessor de 13 é:



Chord

Cada nó mantém um ponteiro para o seu nó sucessor

- O sucessor do nó n é $\text{sucessor}(n+1)$
 - O sucessor de 1 é: $\text{sucessor}(2) = 6$
 - O sucessor de 6 é: $\text{sucessor}(7) = 11$
 - O sucessor de 11 é: $\text{sucessor}(12) = 13$
 - O sucessor de 13 é: $\text{sucessor}(14) = 1$

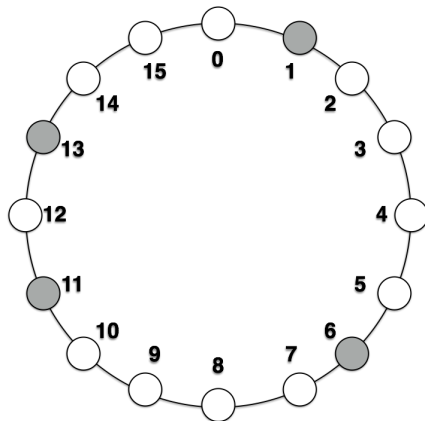


DHT Chord – adicionando dados na rede

- Cada nó é responsável por armazenar um subconjunto de todos os índices (chave,valor)
- O resultado da função hash sobre um dado irá indicar em qual nó essa informação será armazenada

Exemplo

- Chaves no **Nó 1**
 - 14, 15, 0 e 1
- Chaves no **Nó 6**
 - 2, 3, 4, 5 e 6
- Chaves no **Nó 11**
 - 7, 8, 9, 10 e 11
- Chaves no **Nó 13**
 - 12 e 13



DHT Chord – adicionando dados na rede

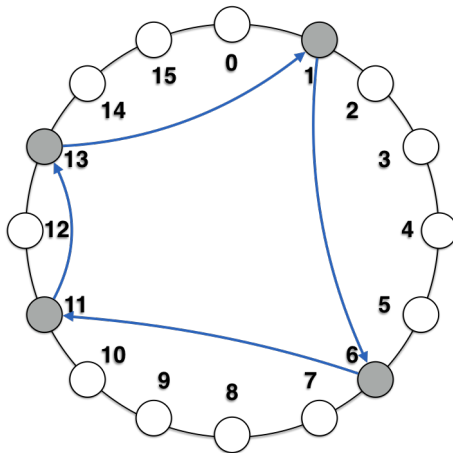
hash("filhão") = 12

hash("filme A") = 3

hash("musica") = 9

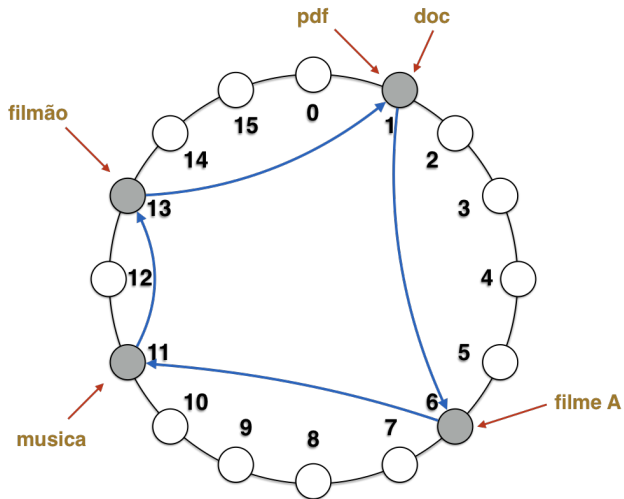
hash("pdf") = 14

hash("doc") = 15



DHT Chord – adicionando dados na rede

hash("filmão") = 12
hash("filme A") = 3
hash("musica") = 9
hash("pdf") = 14
hash("doc") = 15



DHT Chord – ingresso de um nó na rede na posição 8

Antes

- Chaves no **Nó 1**
 - 14, 15, 0 e 1
- Chaves no **Nó 6**
 - 2, 3, 4, 5 e 6
- Chaves no **Nó 11**
 - 7, 8, 9, 10 e 11
- Chaves no **Nó 13**
 - 12 e 13

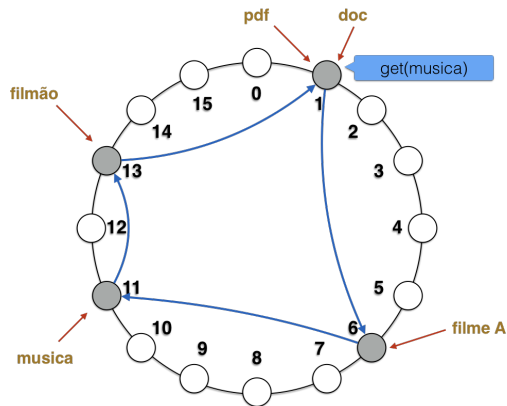
Depois

- Chaves no **Nó 1**
 - 14, 15, 0 e 1
- Chaves no **Nó 6**
 - 2, 3, 4, 5 e 6
- Chaves no **Nó 8**
 - 7 e 8
- Chaves no **Nó 11**
 - 9, 10 e 11
- Chaves no **Nó 13**
 - 12 e 13

DHT Chord – busca por um recurso na rede

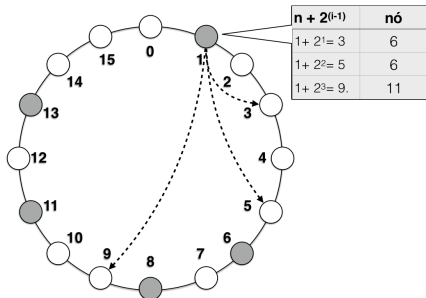
- Calcule o hash(K) e envia mensagem de sucessor para sucessor até atingir o nó sucessor de K
 - Complexidade da busca $O(n)$ (linear) (pior caso com N nós)

- Nó 1 busca por "música"
- $\text{hash}(\text{"musica"}) = 9$
- Busca percorre os nós 1, 6 e 11
- Nó 11 responde para nó 1



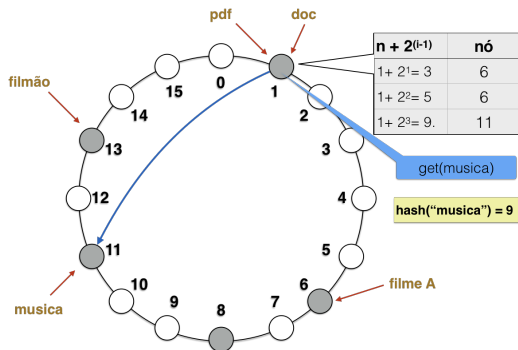
DHT Chord - otimizando buscas com *finger table*

- Cada nó possui *Finger table* (tabela de roteamento) que contém ponteiro para *sucessores*($n + 2^{i-1}$), onde $i = 1 \dots j$ e sendo j o número de linhas na finger table
 - $\log_2(N)$ saltos de qualquer nó para qualquer outro nó
 - Complexidade da busca $O(\log n)$
 - Exemplo: $\log_2(10.000.000) \simeq 23$



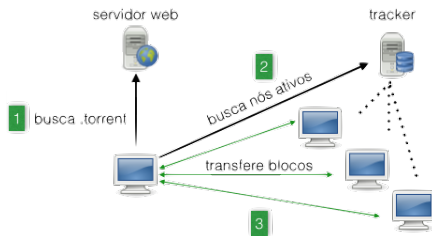
DHT Chord - otimizando buscas com *finger table*

- Cada nó possui *Finger table* (tabela de roteamento) que contém ponteiro para *sucessores*($n + 2^{i-1}$), onde $i = 1 \dots j$ e sendo j o número de linhas na finger table
 - $\log_2(N)$ saltos de qualquer nó para qualquer outro nó
 - Complexidade da busca $O(\log n)$
 - Exemplo: $\log_2(10.000.000) \approx 23$



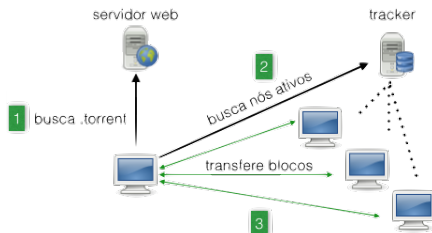
Híbrida: Combina centralizada com descentralizada

- Nó central mantém um índice sobre os demais nós ativos na rede e os recursos disponibilizados por esses
- Ex: **Bittorrent**
 - Arquivos são fracionados em blocos e esses são compartilhados
 - Requisitante obtém blocos sem ordem específica
 - **Arquivo .torrent** contém informações sobre o **tracker**
 - **Tracker** – servidor que mantém a lista de nós ativos para um determinado .torrent



Híbrida: Combina centralizada com descentralizada

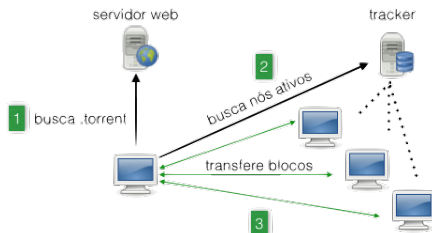
- Nó central mantém um índice sobre os demais nós ativos na rede e os recursos disponibilizados por esses
- Ex: **Bittorrent**
 - Arquivos são fracionados em blocos e esses são compartilhados
 - Requisitante obtém blocos sem ordem específica
 - **Arquivo .torrent** contém informações sobre o **tracker**
 - **Tracker** – servidor que mantém a lista de nós ativos para um determinado .torrent



- Onde está ponto crítico de falha ou gargalo de desempenho?

Híbrida: Combina centralizada com descentralizada

- Nó central mantém um índice sobre os demais nós ativos na rede e os recursos disponibilizados por esses
- Ex: **Bittorrent**
 - Arquivos são fracionados em blocos e esses são compartilhados
 - Requisitante obtém blocos sem ordem específica
 - **Arquivo .torrent** contém informações sobre o **tracker**
 - **Tracker** – servidor que mantém a lista de nós ativos para um determinado .torrent



- Onde está ponto crítico de falha ou gargalo de desempenho?
 - R: trackers

Híbrida – Ex: Bittorrent

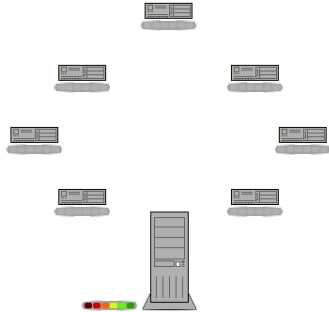


Figura: *

Fonte: wikipedia

Motivo de sucesso

- Encontrou uma forma para acabar com nós egoístas
- Um nó P ao notar que um nó Q está pegando muito mais do que cedendo, então P diminui sua taxa de envio para Q

Híbrida – Ex: Bittorrent

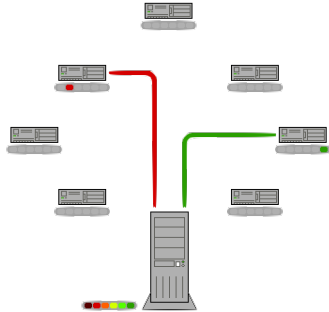


Figura: *

Fonte: wikipedia

Motivo de sucesso

- Encontrou uma forma para acabar com nós egoístas
- Um nó P ao notar que um nó Q está pegando muito mais do que cedendo, então P diminui sua taxa de envio para Q

Híbrida – Ex: Bittorrent

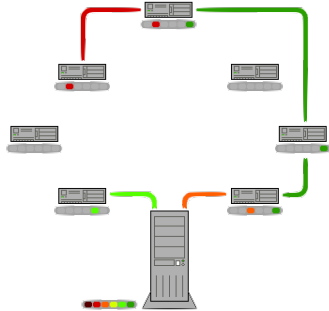


Figura: *

Fonte: wikipedia

Motivo de sucesso

- Encontrou uma forma para acabar com nós egoístas
- Um nó P ao notar que um nó Q está pegando muito mais do que cedendo, então P diminui sua taxa de envio para Q

Híbrida – Ex: Bittorrent

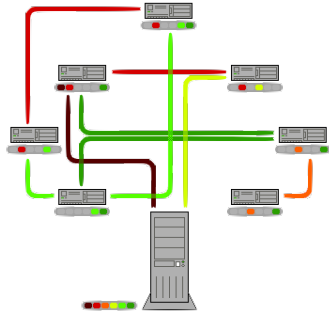


Figura: *

Fonte: wikipedia

Motivo de sucesso

- Encontrou uma forma para acabar com nós egoístas
- Um nó P ao notar que um nó Q está pegando muito mais do que cedendo, então P diminui sua taxa de envio para Q

Híbrida – Ex: Bittorrent

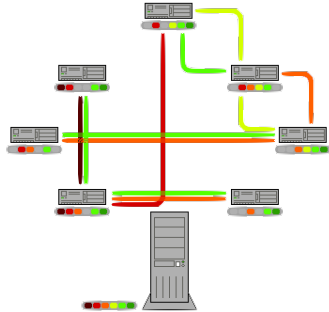


Figura: *

Fonte: wikipedia

Motivo de sucesso

- Encontrou uma forma para acabar com nós egoístas
- Um nó P ao notar que um nó Q está pegando muito mais do que cedendo, então P diminui sua taxa de envio para Q

Híbrida – Ex: Bittorrent

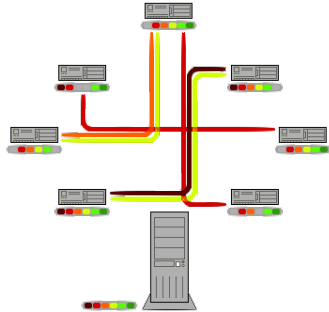


Figura: *

Fonte: wikipedia

Motivo de sucesso

- Encontrou uma forma para acabar com nós egoístas
- Um nó P ao notar que um nó Q está pegando muito mais do que cedendo, então P diminui sua taxa de envio para Q

Híbrida – Ex: Bittorrent

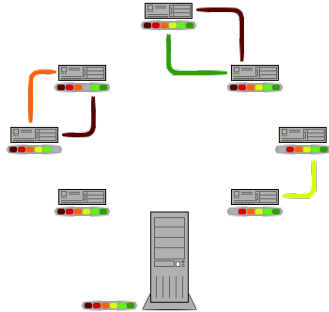


Figura: *

Fonte: wikipedia

Motivo de sucesso

- Encontrou uma forma para acabar com nós egoístas
- Um nó P ao notar que um nó Q está pegando muito mais do que cedendo, então P diminui sua taxa de envio para Q

Híbrida – Ex: Bittorrent

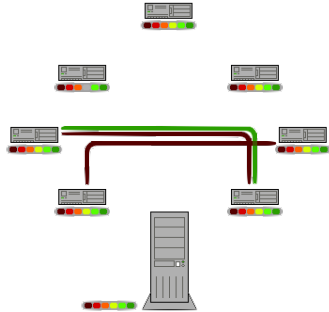


Figura: *

Fonte: wikipedia

Motivo de sucesso

- Encontrou uma forma para acabar com nós egoístas
- Um nó P ao notar que um nó Q está pegando muito mais do que cedendo, então P diminui sua taxa de envio para Q

Híbrida – Ex: Bittorrent

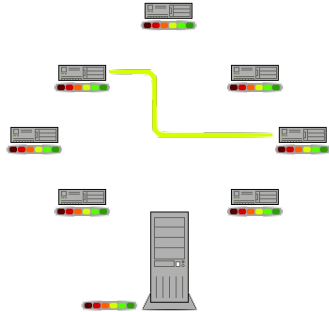


Figura: *

Fonte: wikipedia

Motivo de sucesso

- Encontrou uma forma para acabar com nós egoístas
- Um nó P ao notar que um nó Q está pegando muito mais do que cedendo, então P diminui sua taxa de envio para Q

Híbrida – Ex: Bittorrent

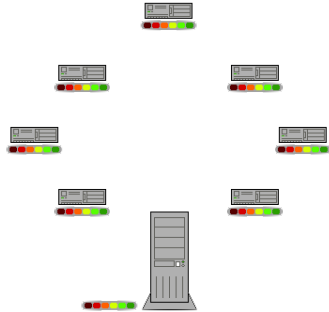


Figura: *

Fonte: wikipedia

Motivo de sucesso

- Encontrou uma forma para acabar com nós egoístas
- Um nó P ao notar que um nó Q está pegando muito mais do que cedendo, então P diminui sua taxa de envio para Q

Leitura Obrigatória

Leitura Obrigatória

- Contêineres de software
 - www.redhat.com/pt-br/topics/containers/whats-a-linux-container
 - www.redhat.com/pt-br/topics/containers/what-is-docker
- Microserviços
 - www.redhat.com/pt-br/topics/microservices/what-are-microservices
- *The Twelve-Factor App*
 - https://12factor.net/pt_br/

Aulas baseadas em



TANENBAUM, Andrew S.; STEEN, Maarten van
Sistemas Distribuidos: Princípios e paradigmas

- Veja Capítulo 2 do livro *Distributed Systems* 3rd edition